

The response consists of `subject`, `to`, and `attachments` as top-level fields (not all properties are shown, for brevity). The `attachments` object has further fields encapsulated as properties with the appropriate fields and their definitions. When we fetch the mapping (`GET emails/_mapping`), while all other fields show their associated data types, `attachments` don't: by default, Elasticsearch infers the `object` type for inner objects.

Next, let's index a document.

Listing 4.25 Indexing an email document

```
PUT emails/_doc/1
{
  "to:":"johndoe@johndoe.com",
  "subject":"Testing Object Type",
  "attachments":{
    "filename":"file1.txt",
    "filetype":"confidential"
  }
}
```

Now that we have primed our `emails` index with one document, we can issue a simple `search` query on the inner object fields to fetch relevant documents (and prove our point).

Listing 4.26 Searching for emails with an attachment file

```
GET emails/_search
{
  "query": {
    "match": {
      "attachments.filename": "file1.txt"
    }
  }
}
```

This query returns the document from our store because the filename matches our document. Note that we use a `term` query on the `keyword` field as we wish to match an exact field value (file1.txt).

While object types are pretty straightforward, they have one limitation: inner objects are *flattened* out and not stored as individual documents. The downside of this action is that the relationship is lost between the objects indexed from an array. Let's review this limitation using a detailed example.